

MODULE 02 · DOMAIN 1 - CONTEXT AND RECONCILIATION

Reconciliation

Finding where the documentation and the deployed code stopped agreeing.

DOMAIN 1 of 5 MODULE 2 of 11 PROMPTS 3

A mapping document is not the specification. It is a **photograph** of one.

Why drift is the default state

This is structural, not a failure of diligence

- ◆ **Documents are written once.** At go live, to get sign off. There is no trigger to revisit them.
- ◆ **Code changes continuously.** Every fix, every enhancement, every source migration.
- ◆ Nobody is slacking. The workflow simply never had a reconciliation step.
- ◆ It never needed one, because the engineer writing the query already knew the document was wrong.

THE REAL CAUSE

Feeding documents to a model removes the person who carried that knowledge.

What each artifact actually knows

THE DOCUMENT KNOWS

Intent

- Why a rule exists at all
- What the business meant by a term
- Which decisions were deliberate
- What was ruled out and why

THE CODE KNOWS

Behaviour

- What actually runs tonight
- Every change since go live
- The literals and codes in force
- Nothing about why any of it is there

THE TIEBREAKER

Neither is authoritative on its own. Reconciliation is how you find out where they diverge.

Three prompts, in this order

01

Diff

Where both sources describe the same thing and say different things.

02

Literals

Where the values inside a matching rule have drifted apart.

03

Orphans

Where only one source mentions something at all.

IN PRACTICE

Run them separately. A combined prompt returns a shorter, worse answer to all three.

The document to code diff.

The output schema you demand

COLUMN	WHY IT IS THERE
Attribute	Anchors every row to something checkable
Document says	Verbatim. Not a paraphrase
Code does	Verbatim. Not a paraphrase
Type	SOURCE, JOIN, FILTER, TRANSFORM, NAME or ABSENT
Confidence	HIGH if unambiguous, LOW if it depends on missing information

THE POINT

Specify the shape or you get prose, and prose about a mapping document is always reassuring.

What you actually type

First half. The framing and the output shape.

```
You are reconciling a data warehouse mapping document against  
the stored procedure that was actually deployed. The document  
was written at go live and has not been maintained. The code  
has changed since. Assume both may be wrong.
```

```
SCOPE: <target table name> only. Ignore all other tables.
```

```
Produce a table with exactly these columns:
```

```
| Attribute | Document says | Code does | Type | Confidence |
```

THE TWO LOAD-BEARING LINES

Naming the scope and the columns is what separates this from a request for a summary.

The rules that make it work

Second half. Paste both halves as one message.

Rules:

- One row per attribute where the two disagree. No matches.
- Quote both sides verbatim, not your paraphrase.
- Type is one of: SOURCE, JOIN, FILTER, TRANSFORM, NAME, ABSENT.
- Confidence is HIGH if unambiguous, LOW if it depends on something not in either file.

After the table, add a section headed CANNOT VERIFY:
anything the document references that is not present in the
material I gave you. Be specific about what is missing.

Do not recommend fixes. Do not summarise.

USE IT AS WRITTEN

Every line exists because dropping it produced a worse answer.

Four design decisions in that prompt

- ◆ **Verbatim, not paraphrase.** A paraphrase of two different things converges on what they share.
- ◆ **One target table per run.** Seven sheets at once gives the same output spread thinner.
- ◆ **Do not list matches.** Otherwise four findings hide among forty non findings.
- ◆ **No recommendations.** A model asked to fix things shades its findings toward what it can fix.

THE POINT

Every one of these is a line in the prompt. None of them is optional.

What it found on the parts usage fact

Five disagreements on one target table

- ◆ **Source schema.** Document says one view layer. Code reads a different one.
- ◆ **Consumption formula.** Document says a simple sum. Code is issues minus returns.
- ◆ **Part hash key.** Document omits the source system key and the upper case wrapper.
- ◆ **Part type filter.** Document and code disagree by a single character.

THE RESULT

Ten minutes of runtime, on a document that had drifted for two years.

The filter that only half applies

Backlog load. Read the last two lines together.

```
01 WHERE NVL(p mats.stloc, '') = '1000'
02       OR (NVL(p mats.stloc, '') = '2000'
03           AND NVL(p mat.ptyp, '') IN ('ZRPR', 'ZCON'))
04       AND NVL(p mats.chgtype, '') <> 'D'
05
06 -- AND binds tighter than OR, so this parses as:
07 --   stloc='1000' OR ( 2000-branch AND not-deleted )
08 -- The delete filter never applies to location 1000.
```

HOW YOU KNOW

Both sibling procedures parenthesise this correctly. That is what establishes intent.

The section headed **cannot verify** is the reason this prompt exists.

What the diff could not resolve

- ◆ It flagged that the consumption formula differs. Correctly.
- ◆ It could not tell you which version is right.
- ◆ The code filters on sixteen movement type codes that appear in neither file.
- ◆ A good run says exactly that, and names what is missing.

THE POINT

It did not solve the problem. It turned an invisible problem into a specific question.

Three failures worth causing on purpose

Break the prompt deliberately so you recognise the symptom

- ◆ **Drop the output schema.** You get a paragraph of reassurance.
- ◆ **Drop the word verbatim.** Two different formulas get paraphrased into the same sentence.
- ◆ **Ask for fixes too.** The cannot verify section vanishes, because nothing in it is actionable.

WATCH FOR

The third one is the dangerous failure. You lose the finding that mattered most.

The literal sweep.

What you actually type

Shorter than it looks. The third paragraph is the one people leave out.

```
List every hard coded literal used in a filter, join, or CASE
condition across all procedures in this file. Include quoted
strings, numeric codes and date constants. Exclude literals
used only as output values or default assignments.
```

```
Group the output BY VALUE, not by procedure. For each value:
  Value | Column compared against | Procedures | Context
```

```
Then add NEAR MISSES: any pair of values above that differ by
one character, a transposition or a case change, AND are
compared against the same column.
```

```
Then add MULTI-LIST VALUES: any value appearing in more than
one list compared against the same column in one statement.
```

```
Do not evaluate correctness. Report only.
```

DO NOT SKIP THE THIRD BLOCK

Drop the last section and movement type 262 survives every pass you will ever run.

The one design decision that matters

GROUPED BY PROCEDURE

The obvious way

- One section per stored procedure
- Matches how you read the file
- Two similar codes end up pages apart
- Nothing connects them

GROUPED BY VALUE

The way that works

- One row per distinct literal
- Sorted, so near matches sit together
- The problem is visible before you read
- Costs nothing extra to ask for

THE TIEBREAKER

Group by value and the defect becomes a visual pattern rather than a reading exercise.

What the sweep returned

VALUE	COMPARED AGAINST	APPEARS IN
ZRPS	part type	Parts usage load
ZRPR	part type	Part site load, backlog load
1000	stock location	All three loads
2000	stock location	All three loads
262	movement type	Issue list AND return list

THE FINDING

One code was corrected in a single procedure eighteen months ago. Two others were missed.

The value that appears in both lists

- ◆ Movement type 262 is in the issue list and in the return list.
- ◆ It gets added, then subtracted. It contributes nothing.
- ◆ Not a near miss. Not a document disagreement. Invisible to both other prompts.
- ◆ One extra section in the prompt catches it. Ask for values appearing in more than one list.

WHY IT HIDES

The query runs. The number is wrong. Nothing about the output would ever tell you.

One value, two lists

Consumption is issues minus returns. Watch movement type 262.

```
01 SUM(CASE WHEN mvttyp IN
02      ('261','262','291','301','311','321','601','641','961')
03      THEN qty ELSE 0 END)          -- issues
04 -
05 SUM(CASE WHEN mvttyp IN
06      ('262','292','302','312','322','602','642')
07      THEN qty ELSE 0 END)          -- returns
08
09 -- 262 is added, then subtracted. It contributes nothing.
```

THE REAL PROBLEM

Sixteen codes, and not one of them appears in the mapping document or the spec.

Where the literal sweep breaks

- ◆ **Dynamic SQL is invisible.** Anything assembled at runtime will not appear.
- ◆ The model will not tell you it looked and found nothing.
- ◆ If the codebase uses execute statements, say so and expect an incomplete sweep.
- ◆ Incomplete and known is fine. Incomplete and assumed complete is not.

THE BLIND SPOT

Know the blind spot, because the output looks identical either way.

The orphan sweep. Run it in **both** directions.

What you actually type

The authority line in paragraph two prevents a silent guess.

```
Compare the columns in the mapping document against the columns
actually written by the stored procedure, for <target table>.
```

```
Treat the procedure's INSERT column list as the authority for
what the code writes. Treat the document's Target Attribute
column as the authority for what was specified.
```

```
Produce three lists:
```

```
  IN CODE, NOT IN DOCUMENT - and what populates each.
```

```
  IN DOCUMENT, NOT IN CODE.
```

```
  WRITTEN BUT NEVER POPULATED - literal NULL or constant.
```

```
Match on column name only. Do not infer that two differently
named columns are the same thing. If you suspect a rename,
list it in both places and say so.
```

WHY THE LAST LINE EXISTS

A guessed rename stated as fact closes a question that should stay open.

Three lists, not two

- ◆ **In document, not in code.** Everyone checks this direction.
- ◆ **In code, not in document.** Almost nobody checks this one.
- ◆ **Written but never populated.** Present in the insert, always null or constant.
- ◆ **Do not infer renames.** A guessed rename closes an investigation that should stay open.

THE POINT

Name the insert column list as the authority, or the model picks between three conflicting lists silently.

What the orphan sweep found

- ◆ **Six measures in code, in no document.** Issue and return totals for each time window.
- ◆ Those six are the intermediate values that reveal how consumption is really calculated.
- ◆ **One column written, never populated.** A slow mover flag, listed as deferred in the spec.
- ◆ The second finding is not a defect. It is still worth knowing.

THE COST

Somebody added those six columns to validate the net figure. That decision is recorded nowhere.

Which side is right?

PARTS USAGE FACT

The document is wrong

- Document builds a key with a plus sign
- Code concatenates properly
- Every other sheet agrees with the code
- Document is the outlier

PART SITE DIMENSION

The code is wrong

- Document concatenates properly
- Code builds a value with a plus sign
- Same mistake, opposite side
- Code is the outlier

THE TIEBREAKER

Reconciliation tells you where they disagree. It does not tell you who is correct.

Which of these is the outlier?

Top pair from the document. Bottom pair from the code.

```
01 -- Mapping document, parts usage fact:
02 f_fnv_hash(source_system_part_number + source_system_site_code)
03 -- Deployed code, parts usage fact:
04 f_fnv_hash(v_SourceSystemKey || UPPER(partno) || UPPER(site))
05
06 -- Mapping document, part site dimension:
07 UPPER(TRIM(stloc) || '|' || TRIM(plntyp))
08 -- Deployed code, part site dimension:
09 UPPER(TRIM(pmats.stloc) + '|' + TRIM(pmats.plntyp))
```

THE TIEBREAKER

In Redshift the plus sign is not string concatenation. Both uses of it are wrong.

A planner reports that consumption for one site looks about a third too low. The pipeline has not failed in months and no alerts have fired.

You have the mapping document, the deployed stored procedures, and no access to the source system. The original developer left last year.

What do you do first?

- ☐ A Ask a model to rewrite the consumption query from the mapping document
- ☐ B Run the document to code diff on the consumption fact table
- ☐ C Query the source tables directly and compare row counts
- ☐ D Ask a model to review the stored procedure for bugs

◆ PAUSE AND DECIDE

What do you do first?

- A

 Rewrite from the mapping document
The document is the thing you have not verified. Rewriting from it reproduces whatever is stale in it.
- B

 Run the document to code diff
Cheapest step that tells you whether the document you are about to trust is still true.
- C

 Compare source row counts
Reasonable, but you said you have no source access. Also slower than reading two files you already hold.
- D

 Ask for a bug review
A generic review finds generic problems. It has no way to know what the code was supposed to do.

THE SIGNAL

Nothing has failed and no alert fired. That is the signal that the defect is in a definition, not in the plumbing.

The diff returns a clean table of six disagreements, and a cannot verify section saying the movement type codes used by the calculation appear in neither file.

Your deadline is tomorrow. The Materials Management team who own the code register are two time zones away.

How should you proceed?

- ☐ A Ask the model to infer the correct movement types from the code
- ☐ B Treat the deployed code as authoritative and document what it does
- ☐ C Send the specific question to Materials Management and proceed on the rest
- ☐ D Rerun the diff with a longer, more detailed prompt

◆ PAUSE AND DECIDE

How should you proceed?

A Infer the codes from the code

The code is the only evidence, so this just restates it with false confidence attached.

C Send the question, proceed on the rest

The cannot verify section produced one specific, answerable question. Asking it is the work.

B Treat the code as authoritative

Tempting and half right. But we saw one code corrected in only one of three procedures, so the code disagrees with itself.

D Rerun with a better prompt

No prompt recovers information that is not in the context. This is the lesson of the module.

THE SIGNAL

The prompt already did its job. It told you exactly which question to ask and who to ask.

A colleague argues that reconciliation is wasted effort. The documents are always stale, so the team should stop maintaining them and read the code when they need the truth.

They point out that the code is version controlled, always current, and cannot lie about what it does.

What is the strongest argument against this?

- ☐ A Auditors require the documentation to exist
- ☐ B Reading code is slower than reading a document
- ☐ C The code cannot tell you which behaviours are deliberate
- ☐ D Version control history explains why changes were made

◆ PAUSE AND DECIDE

What is the strongest argument against this?

- A

 Auditors require it
True in many places and completely unpersuasive. It argues for a document nobody reads.
- B

 Reading code is slower
Often true, but speed is not the issue. A slow correct answer beats a fast wrong one.
- C

 Code cannot tell you what is deliberate
The plus sign defect and the intended concatenation look identical in code. Only intent separates them.
- D

 History explains the why
Commit messages rarely carry business reasoning, and a squashed migration erases what little there was.

THE SIGNAL

Every defect in this module looks exactly like a deliberate design decision until something outside the code says otherwise.

You run the orphan sweep and it returns four columns present in the code and absent from the document. Three are clearly internal audit fields. The fourth is a business measure.

You have limited time and a model that will happily produce a plausible definition for any of them.

What is the right next step for the fourth column?

- ☐ A Ask the model to write a definition from how the code populates it
- ☐ B Add it to the mapping document as undocumented, and raise it
- ☐ C Leave it out, since nothing downstream is asking for it
- ☐ D Ask the model whether the column is important

◆ PAUSE AND DECIDE

What is the right next step for the fourth column?

- A

 Generate a definition from the code
Produces a mechanical description of the calculation. It cannot recover why anyone wanted the measure.
- B

 Record it as undocumented and raise it
Preserves the finding, marks it as unresolved, and routes it to someone who can answer.
- C

 Leave it out
Nothing asking for it today is not evidence. Somebody built it deliberately and stopped being asked.
- D

 Ask if it is important
The model has no basis for that judgement and will answer anyway. This is the worst option here.

THE SIGNAL

A generated definition looks exactly like a real one, and it silently closes the question.

Take this to work tomorrow

- ◆ Pick one table you are about to change. Run the diff on it before you touch anything.
- ◆ Run the literal sweep across your whole procedure file. It costs one prompt.
- ◆ Whatever lands in cannot verify becomes a message to a named person, not a to do item.
- ◆ Do not fix anything in the same session. Findings and fixes are separate passes.

THE PAYOFF

Three prompts, ten minutes, on documents nobody has opened in two years.

Reconciliation produces a list of **questions**,
not answers.